



UNIVERSIDADE FEDERAL DO MARANHÃO
CENTRO DE CIÊNCIAS EXATAS E TECNOLÓGICAS
CURSO DE CIÊNCIA DA COMPUTAÇÃO
ESTRUTURA DE DADOS II

MIKA MARQUES DOS SANTOS JÚNIOR

Relatório da Atividade prática - Unidade 2
Árvore Rubro-negra e Tabela Hash

São Luís - MA
2026

MIKA MARQUES DOS SANTOS JÚNIOR

Relatório da Atividade prática - Unidade 2

Árvore Rubro-negra e Tabela Hash

Relatório apresentado como requisito para obtenção parcial de nota na disciplina Estrutura de Dados 2, do curso Ciência da Computação da Universidade Federal do Maranhão, sob orientação do professor João Dallyson Sousa de Almeida.

São Luís - MA

2026

SUMÁRIO

1 INTRODUÇÃO.....	4
2 QUESTÃO 1 - ÁRVORE RUBRO-NEGRA.....	4
2.1 Classe NoRN<AnyType>.....	4
2.2 Rotações e alterações.....	5
2.3 Inserção e alterações.....	6
2.4 Remoção e alterações.....	7
2.5 Casos de teste.....	8
2.5.1 Arquivo dados_800.....	8
2.5.2 Arquivo dados_500k.....	9
2.5.3 Arquivo dados_1M.....	9
3 QUESTÃO 2 - TABELA HASH.....	10
3.1 Função Hash.....	10
3.2 Tratamento de colisões.....	10
3.3 Redimensionamento.....	11
3.4 Casos de teste.....	12
3.4.1 Arquivo dados_800.....	12
3.4.2 Arquivo dados_500k.....	13
3.4.3 Arquivo dados_1M.....	13
4 CONCLUSÃO.....	14

1 INTRODUÇÃO

Este relatório tem como objetivo de registrar os casos de teste de duas Estruturas de Dados implementadas na atividade prática: Uma Árvore Rubro-Negra, na qual cada nó da árvore tem referência explícita ao irmão, e uma Tabela Hash com uso de Generics, cuja a função hash utiliza o método da multiplicação, e tratamento de colisões é realizado com hash quadrático.

As implementações dos métodos em ambos os TAD's foram baseados nos algoritmos apresentados nas aulas 9, 10, 12 e 13, além do uso do livro: *“Estruturas de dados, algoritmos, análise da complexidade e implementações em JAVA e C/C++”* de Ana Ascencia e Graziela Araújo.

2 QUESTÃO 1 - ÁRVORE RUBRO-NEGRA

Conforme solicitado na questão, será demonstrado a solução para a modificação da classe `NoRN<AnyType>` e dos métodos de `LEFT_ROTATE`, `RIGHT_ROTATE`, `RB_INSERT`, `RB_INSERT_FIXUP`, `RB_DELETE` e `RB_DELETE_FIXUP` e os casos de teste usados.

2.1 Classe `NoRN<AnyType>`

A classe `NoRN` define a estrutura de cada nó na árvore, além dos campos originais apresentados em aula, foi acrescentado a referência explícita *irmao*, usado para manter a referência do irmão de um nó do mesmo pai. A classe de nó foi criada como uma classe estática dentro da classe da árvore Rubro-Negra.

```
public class RubroNegra<AnyType> {
    private static class NoRN<AnyType> {
        AnyType elemento;
        NoRN<AnyType> pai, esquerdo, direito, irmao; //Referência explícita do irmao
        boolean cor;
        int codigo;

        NoRN(int codigo, AnyType e) {
            this(codigo, e, pai: null, irmao: null, esq: null, dir: null);
        }

        NoRN(int codigo, AnyType e, NoRN<AnyType> pai, NoRN<AnyType> irmao, NoRN<AnyType> esq,
        NoRN<AnyType> dir) {
            this.codigo = codigo;
            this.elemento = e;
            this.esquerdo = esq;
            this.direito = dir;
            this.pai = pai;
            this.irmao = irmao;
            this.cor = RubroNegra.RED;
        }
    }
}
```

Figura 1 - Código da classe interna da Árvore Rubro-Negra.

2.2 Rotações e alterações

Os métodos LEFT_ROTATE e RIGHT_ROTATE foram os mais afetados pela implementação da referência do nó irmão, pois é necessário manter a consistência da árvore após as rotações necessárias:

Na rotação, o nó y substitui a posição do nó x, então é necessário salvar o irmão de x para atribuí-lo posteriormente ao irmão y. Além disso, no caso do método LEFT_ROTATE, na passagem do filho esquerdo de y para ser o filho direito de x, é necessário atualizar a referência dos irmãos dos nós filhos de x:

```
NoRN<AnyType> y = x.direito;
NoRN<AnyType> z = x.irmao; //Salva irmao de x
x.direito = y.esquerdo;

//atribuição dos novos irmaos dos filhos de x
if (y.esquerdo != this.Nil) {
    y.esquerdo.pai = x;
    y.esquerdo.irmao = x.esquerdo;
}

if (x.esquerdo != this.Nil) {
    x.esquerdo.irmao = x.direito;
}
```

Figura 2 - Alteração 1 do método LEFT_ROTATE para suporte a referência do irmão

Após o processo de rotação, é necessário atualizar a referência de irmão dos filhos do nó y, além de atualizar a referência de irmão do próprio nó y:

```
//atualizacao dos irmãos dos nós filhos de y
x.irmao = y.direito;
if (y.direito != this.Nil) {
    y.direito.irmao = x;
}

//atualização do irmao de y
y.irmao = z;
if (z != this.Nil) {
    z.irmao = y;
}
```

Figura 3 - Alteração 2 do método LEFT_ROTATE para suporte a referência do irmão

As figuras exibidas são as modificações realizadas no método de rotação para esquerda (LEFT_ROTATE), o método RIGHT_ROTATE segue quase o mesmo algoritmo, apenas substituindo os termos “esquerdo” por “direito” e vice-versa, por conta da simetria dessas funções.

2.3 Inserção e alterações

O método RB_INSERT foi adaptado para criar e manter a referência do irmão no momento da inserção. Sendo y o nó pai do nó inserido (z):

1. Se o pai é Nil, ou seja, uma árvore vazia, então z se torna a nova raiz, e consequentemente não tem irmão.
2. Caso o filho z for o esquerdo, então seu irmão é o nó à direita, e também, caso exista, nó da direita recebe como irmão o nó z.
3. Simétrico ao caso 2.

```

z.pai = y;
if (y == this.Nil) {
    this.root = z;
    z.irmao = this.Nil;
} else if (z.codigo < y.codigo) {
    y.esquerdo = z;
    z.irmao = y.direito;
    if (y.direito != this.Nil) {
        y.direito.irmao = z;
    }
} else {
    y.direito = z;
    z.irmao = y.esquerdo;
    if (y.esquerdo != this.Nil) {
        y.esquerdo.irmao = z;
    }
}

```

Figura 4 - Adaptação de RB_INSERT para suporte a referência do irmão

Já o método RB_INSERT_FIXUP é o que mais se aproveita do uso da referência ao irmão, pois para acessar o tio de um nó não é mais necessário subir para o avô para acessar o tio, basta acessar o irmão do pai:

```

(z.pai == z.pai.pai.esquerdo) {
  NoRN<AnyType> y = z.pai.irmao;
  if (y.cor == RubroNegra.RED) { //caso 1

```

Figura 5 - Uso da referência irmão para acessar o tio do nó z

2.4 Remoção e alterações

O método RB_DELETE foi alterado para se adaptar ao uso do irmão. Sendo y o nó a ser removido, e x o nó que substituirá a posição de y, é necessário que x herda o irmão que y tinha:

```

if (y.pai == this.Nil) {
  this.root = x;
} else if (y == y.pai.esquerdo) {
  y.pai.esquerdo = x;
  x.irmao = y.irmao;
  if (x.irmao != this.Nil) {
    x.irmao.irmao = x;
  }
} else {
  y.pai.direito = x;
  x.irmao = y.irmao;
  if (x.irmao != this.Nil) {
    x.irmao.irmao = x;
  }
}

```

Figura 6 - Adaptação no método RB_DELETE para tratar referência do irmão.

O método RB_DELETE_FIXUP se beneficia com a nova referência, pois para tratar os casos de uma remoção de um nó preto, é necessário acessar o irmão do nó atual. Na antiga implementação, era necessário subir para o pai e acessar o irmão, agora basta somente acessar diretamente o irmão.

```

NoRN<AnyType> w = x.irmao;

```

Figura 7 - Adaptação no método RB_DELETE_FIXUP

2.5 Casos de teste

A fim de realizar testes e comprovar o uso correto da Árvore Rubro-Negra, foram implementados métodos extras de impressão, na modalidade Pré-ordem, pós-ordem e Simétrica. Além disso, foi criada uma classe teste chamada Funcionario:

```
public class Funcionario {
    String nome;
    int idade;
    float salario;

    public Funcionario(String nome, int idade, float salario) {
        this.nome = nome;
        this.idade = idade;
        this.salario = salario;
    }

    @Override
    public String toString() {
        return String.format(format: "%-25s | Idade: %2d | R$ %.2f",
            nome, idade, salario);
    }
}
```

Figura 8 - Classe Funcionario usada para teste.

Por meio de um script disponibilizado no repositório, o arquivo *Gerador.java*, foram criados 3 arquivos de texto contendo informações pseudo-aleatórias de funcionários para submeter para a Árvore Rubro-Negra. Foi dedicada uma parte do código do método main que adiciona os dados dos arquivos de forma consecutiva até finalizar a leitura de todo arquivo. Além do uso do método *System.currentTimeMillis()* para calcular o tempo das operações.

2.5.1 Arquivo dados_800

Esse arquivo contém 800 dados de diferentes funcionários, diferenciados pelo seu código de identificação usado na classe NoRN:

1. Inserção dos 800 elementos: tempo em cerca de 8 ms

```
Deseja carregar a árvore com dados? (1 -> sim | 0 -> não)
1
Inserções bem sucedidas
Tempo: 8 ms
```

2. Busca de um elemento específico: tempo aproximado a 0 ms

```
Digite o código do funcionario a buscar:
800
Tempo: 0 ms
Isabela Vieira | Idade: 37 | R$ 12100,54
```

3. Remoção de um elemento: tempo aproximado a 0 ms


```

Digite o código do funcionario a remover:
800
Tempo: 0 ms
Remoção bem sucedida:
Isabela Vieira          | Idade: 37 | R$ 12100,54

```

2.5.2 Arquivo dados_500k

Esse arquivo contém os dados de 500 mil funcionários fictícios pseudo-aleatórios:

1. Inserção dos 500 mil elementos: tempo em cerca de 752 ms

```

Deseja carregar a árvore com dados? (1 -> sim | 0 -> não)
1
Inserções bem sucedidas
Tempo: 752 ms

```

2. Busca de um elemento específico: tempo aproximado a 0 ms

```

Digite o código do funcionario a buscar:
500000
Tempo: 0 ms
Ricardo Goncalves      | Idade: 24 | R$ 18525,73

```

3. Remoção de um elemento: tempo aproximado a 0 ms

```

Digite o código do funcionario a remover:
500000
Tempo: 0 ms
Remoção bem sucedida:
Ricardo Goncalves      | Idade: 24 | R$ 18525,73

```

2.5.3 Arquivo dados_1M

Para fazer um teste em larga escala, esse arquivo contém 1 milhão de dados:

1. Inserção dos 1 milhão de elementos: tempo em cerca de 1 segundo e 700ms

```

Deseja carregar a árvore com dados? (1 -> sim | 0 -> não)
1
Inserções bem sucedidas
Tempo: 1747 ms

```

2. Busca de um elemento específico: tempo aproximado a 0 ms

```

Digite o código do funcionario a buscar:
1000000
Tempo: 0 ms
Olivia Costa           | Idade: 33 | R$ 19553,04

```

3. Remoção de um elemento: tempo aproximado a 0 ms

```

Digite o código do funcionario a remover:
1000000
Tempo: 0 ms
Remoção bem sucedida:
Olivia Costa           | Idade: 33 | R$ 19553,04

```

3 QUESTÃO 2 - TABELA HASH

Conforme solicitado na questão, será demonstrado a função hash com o método da multiplicação, além do tratamento de colisões e o redimensionamento. Ademais, caso o usuário inicialize a tabela sem informar um valor base de tamanho, o tamanho inicial definido foi 997.

3.1 Função Hash

A função hash da tabela foi implementada da seguinte forma:

```
/*Método hash pela estratégia da multiplicação que recebe um inteiro como chave*/
private int hash(int k) {
    if (k < 0) {
        k *= -1;
    }
    double code = this.T * ((k * this.A) % 1);
    return (int) code;
}
```

Nesta implementação, caso a chave seja negativa, ela é invertida para positiva. o valor da constante A segue o valor sugerido na aula 10 slide 25:

```
private final double A = 0.6180339887;
```

3.2 Tratamento de colisões

O tratamento de colisões, conforme dito na questão, foi feito por meio do hash quadrático, sendo implementado de modo independente nos métodos de inserção, busca e remoção:

```
public G search(int key) {
    int i = 1;
    int h = hash(key);

    while (tabela[h].codigo != key && tabela[h].livre != 'L') {
        if (i >= this.T) {
            return null;
        }
        h = (hash(key) + (i * i)) % this.T;
        i++;
    }
}
```

Figura 9 - Parte do tratamento de colisões na busca

```

public boolean insert(int key, G data) {
    int i = 1;
    int h = hash(key);
    hashNode<G> newnode = new hashNode<>(key,data);

    while (tabela[h].livre == '0' && i < this.T) {
        if (tabela[h].codigo == key) {
            return false;
        }
        h = (hash(key) + (i * i)) % this.T;
        i++;
    }
}

```

Figura 10 - Tratamento de colisões na inserção

```

public G remove(int key) {
    int h = hash(key);
    int i = 1;

    while (tabela[h].codigo != key && tabela[h].livre != 'L') {
        if (i >= this.T) {
            return null;
        }
        h = (hash(key) + (i * i)) % this.T;
        i++;
    }
}

```

Figura 11 - Tratamento de colisões na remoção

3.3 Redimensionamento

O redimensionamento do vetor é realizado quando o fator de carga da tabela é igual ou maior que 0,7, ou seja, acima de 70% ocupado. Essa verificação é feita após a inserção de um elemento:

```

if (i < this.T && tabela[h].livre != '0') {
    tabela[h] = newnode;
    this.n++;
    double fatorCarga = (double) this.n / this.T;
    if (fatorCarga >= 0.7) {
        resize();
    }

    return true;
}

```

Figura 12 - Verificação do fator de carga após a inserção de um elemento na tabela

Dentro do método `resize` é criada uma nova instância de um vetor de nós hash, com o tamanho 3 vezes maior que a anterior, e também salva a referência da tabela antiga. Por fim, para todas as posições da tabela antiga indicadas como ocupadas, é realizado um hash para a nova tabela.

```

@SuppressWarnings("unchecked")
private void resize() {
    hashNode<G>[] tabelaAntiga = this.tabela;
    int lastT = this.T;
    this.T *= 3;
    this.tabela = (hashNode<G>[]) new hashNode[this.T];
    this.n = 0;

    for (int i = 0; i < this.T; i++) {
        this.tabela[i] = new hashNode<>();
    }

    for (int j = 0; j < lastT; j++) {
        hashNode<G> elem = tabelaAntiga[j];
        if (elem.livre == '0') {
            insert(elem.codigo, elem.dados);
        }
    }
}

```

Figura 13 - Função de redimensionamento dinâmico da tabela hash

3.4 Casos de teste

Do mesmo modo que a questão 1, os casos de teste foram separados em 3 arquivos de texto para analisar a velocidade de execução dos métodos da estrutura. Também foi usado a classe Funcionario como teste.

3.4.1 Arquivo dados_800

Esse arquivo contém 800 dados de diferentes funcionários, diferenciados pelo seu código de identificação:

1. Inserção dos 800 elementos: tempo em cerca de 8 ms

```

Deseja carregar a árvore com dados? (1 -> sim | 0 -> não)
1
Inserções bem sucedidas
Tempo: 8 ms

```

2. Busca de um elemento específico: tempo aproximado a 0 ms

```

Digite o código do funcionario a buscar:
245
Tempo: 0 ms
Bruno Soares          | Idade: 53 | R$ 14592,87

```

3. Remoção de um elemento: tempo aproximado a 0 ms

```
Digite o código do funcionario a remover:
245
Tempo: 0 ms
Remoção bem sucedida:
Bruno Soares | Idade: 53 | R$ 14592,87
```

3.4.2 Arquivo dados_500k

Esse arquivo contém os dados de 500 mil funcionários fictícios pseudo-aleatórios:

1. Inserção dos 500 mil elementos: tempo em cerca de 485 ms

```
Deseja carregar tabela com dados? (1 -> sim | 0 -> não)
1
Inserções bem sucedidas
Tempo: 485 ms
```

2. Busca de um elemento específico: tempo aproximado a 0 ms

```
Digite o código do funcionario a buscar:
85257
Tempo: 0 ms
Luciana Costa | Idade: 28 | R$ 5946,83
```

3. Remoção de um elemento: tempo aproximado a 0 ms

```
Digite o código do funcionario a remover:
85257
Tempo: 0 ms
Remoção bem sucedida:
Luciana Costa | Idade: 28 | R$ 5946,83
```

3.4.3 Arquivo dados_1M

Para fazer um teste em larga escala, esse arquivo contém 1 milhão de dados:

1. Inserção dos 1 milhão de elementos: tempo em cerca de 1 segundo

```
Deseja carregar tabela com dados? (1 -> sim | 0 -> não)
1
Inserções bem sucedidas
Tempo: 1004 ms
```

2. Busca de um elemento específico: tempo aproximado a 0 ms

```
Digite o código do funcionario a buscar:
952350
Tempo: 0 ms
Gustavo Lopes | Idade: 30 | R$ 14465,95
```

3. Remoção de um elemento: tempo aproximado a 0 ms

```
Digite o código do funcionario a remover:  
952350  
Tempo: 0 ms  
Remoção bem sucedida:  
Gustavo Lopes | Idade: 30 | R$ 14465,95  
1 - Insira um elemento
```

4 CONCLUSÃO

Portanto, a partir dos casos de testes mostrados brevemente, nota-se a eficiência das estruturas de dados Árvore Rubro-Negra e a Tabela Hash, e que, apesar do crescimento da quantidade de dados, o tempo de execução cresce de modo controlado, seguindo o referencial assintótico de cada estrutura, a complexidade logarítmica para Rubro-Negra e a complexidade constante das Tabelas Hash.

A verificação da integridade das propriedades da Árvore Rubro-Negra e os elementos que compõem a tabela hash podem ser visualizados por meio de estratégias de impressão de dados utilizadas nos métodos Main dos arquivos de chamadas. Por fim, esse trabalho tem como objetivo consolidar os conceitos e aprimorar a prática envolta dessas estruturas de dados complexas e eficientes.